

Scala vs. Rust

Integrantes:

- Tomás Bertotto
- Nicolás Cid

Sintaxis y Semantica

En cuanto a la **sintaxis léxica**, Scala es similar a Java y se caracteriza por ser flexible, mientras que Rust se asemeja más a C/C++ y es más estricto. Respecto a la **sintaxis gramatical**, Scala resulta conciso y expresivo, permitiendo omisiones, mientras que Rust es más explícito y estructurado, con uso obligatorio de “;”.

En la **semántica estática**, ambos lenguajes poseen tipado estático y fuerte. Sin embargo, Scala destaca por una inferencia de tipos más potente e implícita, mientras que Rust presenta una inferencia más limitada y explícita, lo que lo hace más predecible.

En los **chequeos en compilación**, Scala se centra en tipos, declaraciones y restricciones generales, mientras que Rust agrega controles más avanzados como ownership, borrowing y mutabilidad, lo que refuerza la seguridad.

En la **semántica dinámica**, Scala se ejecuta sobre la JVM, utiliza recolección de basura (garbage collector) y permite evaluación perezosa (lazy) opcional. Rust, en cambio, genera código nativo, no utiliza garbage collector y emplea evaluación ansiosa.

Por último, en la **gestión de memoria**, como se menciona en el párrafo anterior Scala depende de un sistema automático mediante garbage collector, mientras que Rust utiliza un modelo basado en ownership y borrowing, sin necesidad de recolección de basura.

En líneas generales, la sintaxis léxica, la semántica estática y la facilidad de lectura de ambos lenguajes son similares, con algunas diferencias sutiles. Rust es más estructurado en el chequeo de la sintaxis gramatical y en la inferencia de tipos, evitando declaraciones implícitas. Consideramos que estas características son sumamente positivas; no obstante, le quitan cierta flexibilidad al lenguaje.

Por otro lado, Scala ofrece un enfoque más conveniente y productivo para el desarrollador. Al ejecutarse sobre la JVM y contar con recolección de basura automática, permite abstraerse de los detalles de administración de memoria, reduciendo la posibilidad de errores y simplificando el desarrollo. Además, la posibilidad de utilizar evaluación perezosa de forma opcional brinda mayor flexibilidad para optimizar el rendimiento según el contexto.

En conjunto, estas características hacen que Scala favorezca la rapidez de desarrollo, la mantenibilidad del código y una menor carga cognitiva, en contraste con Rust, que exige un control más explícito y detallado de los recursos.

Constructores fundamentales del lenguaje

En cuanto a las **variables o ligaduras**, ambos lenguajes distinguen entre valores mutables e inmutables. En Scala esto se expresa mediante **val** para valores inmutables y **var** para mutables, mientras que en Rust se utiliza **let** y **let mut** respectivamente.

Respecto a las **estructuras de control**, ambos lenguajes comparten construcciones clásicas como **if**, **while** y **for**, y también permiten el uso de **recursión**. Sin embargo, Rust incorpora además **loop** y **match** como mecanismos adicionales de control de flujo, mientras que en Scala la recursión suele utilizarse de forma más idiomática como alternativa a los bucles tradicionales.

En el caso de las **funciones**, tanto Scala como Rust las tratan como ciudadanos de primera clase. Scala, no obstante, destaca por su soporte de **lambdas** y una inferencia de tipos más integrada, mientras que Rust utiliza **closures** y otros mecanismos equivalentes.

En relación con las **estructuras de datos**, Scala ofrece colecciones como **List**, **Set** y **Map**, que son inmutables por defecto, salvo que se indique lo contrario. Por su parte, Rust dispone de estructuras como **Vec**, **HashMap** y **arrays**, que suelen ser mutables por defecto.

En la **definición de tipos**, Scala cuenta con **class**, **object**, **trait** y **case class**, brindando una gran flexibilidad para modelar distintos conceptos. Rust, en cambio, utiliza **struct**, **enum** y **trait** como sus principales herramientas.

En términos de **modularización**, Scala se organiza mediante **packages**, **objetos** y **traits**, mientras que Rust utiliza **módulos (mod)** y **crates** como unidades principales de organización.

Finalmente, en el **manejo de errores**, Scala emplea excepciones con bloques **try/catch**, mientras que Rust adopta un enfoque basado en tipos como **Result** y **Option**, evitando el uso de excepciones tradicionales.

En conclusión, Rust y Scala no difieren en gran medida en lo que refiere a variables, estructuras de control y funciones. Sin embargo, se observa que Rust presenta constructores más explícitos y controlados, mientras que Scala ofrece un enfoque más abstracto y expresivo. Esto vuelve a poner en evidencia la dicotomía entre la seguridad y el rendimiento que prioriza Rust frente a la productividad que favorece Scala. En este sentido, y en línea con lo mencionado anteriormente, consideramos que la mayor expresividad y nivel de abstracción de Scala lo convierten en una opción más adecuada cuando se busca rapidez de desarrollo y mantenibilidad del código.

Paradigma

Scala está concebido como un lenguaje que **fusiona el paradigma orientado a objetos y el paradigma funcional** en su diseño (p.55, manual de Scala). El lenguaje asegura/implementa características propias del paradigma orientado a objetos asegurando que: **cada valor es un objeto y cada operación es una llamada a un método**, por ejemplo:

```
var ejemplo = 1 + 2
```

Acá, la operación "1 + 2" se traduce a una llamada a un método "+" definido en la clase "Int".

Scala es sumamente **consistente** en este sentido por lo que hasta el mismo valor de las funciones son objetos.

A su vez, Scala admite el **paradigma funcional** bajo dos ideas centrales: las funciones como valores de primera-clase, y la idea de inmutabilidad y transparencia referencial. La primera de ellas establece que las funciones son valores como pueden serlo un entero o un string lo cual habilita devolver funciones, usar funciones como parámetros, entre otras. por ejemplo:

```
var resta: (Int, Int) => Int = (a, b) => a - b
```

```
def aplicar_funcion(f: (Int, Int) => Int, x: Int, y: Int) =  
    f(x,y)
```

(primer caso una funcion sin nombre almacenada en la variable “resta” y en el segundo caso trabajar con una función como parámetro)

La segunda idea trabaja con el concepto de inmutabilidad, se evita modificar la información/valores actuales, por ejemplo: *la modificación de un string “hola” produce un nuevo objeto con el string modificado sin cambiar el objeto inicial cuyo valor es “hola”*. Esto se traduce, claramente, a que los métodos no deben tener efectos colaterales (*transparencia referencial*).

Rust se describe como un lenguaje que contiene **características del paradigma funcional y del paradigma orientado a objetos**. En el primer caso lo hace admitiendo: **clausuras**(función anónima tratada como valor que captura variables del entorno) e **iteradores** (permiten recorrer una estructura e inclusive procesar sus elementos con evaluación perezosa).

Ejemplo **clausura**:

```
fn main() {  
    let x = 4;  
    let equal_to_x = |z| z == x;  
    let y = 4;  
    assert!(equal_to_x(y));  
}
```

(manual de Rust p.306)

La función se almacena de forma anónima (no tiene nombre) en la variable “equal_to_x” donde en compara con el valor de “x” que no está declarado como parámetro sino que la toma de su entorno.

Por otro lado, las **características del paradigma orientado a objetos** en Rust se plantean como:

- Idea de “objeto” como container de información a través structs y enums
- Encapsulamiento: todo es privado por defecto (salvo usando modificador *pub*)
- Herencia simulada a través de traits para abstraer comportamiento

Conclusión para cada caso: Scala hace un trabajo íntegro para con ambos paradigmas, provee constructores y mecanismos lo que se traduce en una facilidad en el uso del lenguaje. Rust en cambio presenta características que se acercan a ambos paradigmas pero sigue siendo un lenguaje orientado fuertemente al paradigma imperativo. Además hay un grado de artificialidad en cómo se moldean los conceptos de objeto y la herencia/polimorfismo en Rust que lo hace mucho más rígido que Scala. Esa simulación para con nociones básicas del paradigma se traduce en un código que no

refleja a simple vista la abstracción y/o semántica del programa que se quiere lograr, lo que empeora la legibilidad fuertemente. Ambos lenguajes no exigen una obligación en el uso de sus paradigmas: los admiten. Teniendo en cuenta lo mencionado anteriormente en tanto a los paradigmas concluimos que Rust tiene una menor flexibilidad y peor legibilidad que Scala.

Tipos soportados y Sistema de Tipos

La **inmutabilidad** en Scala y Rust es el comportamiento por defecto (val y let, respectivamente), aunque ambos permiten la mutabilidad de forma explícita: Scala mediante var y Rust a través de la palabra clave mut.

Scala ofrece typeclasses mediante implicits (given/using), **subtipado y herencia** a través de clases y traits, además de soporte limitado para tipos dependientes y soporte completo para tipos de orden superior. Rust, en cambio, implementa typeclasses mediante traits, pero no posee subtipado ni herencia (solo composición con traits), no soporta tipos dependientes y tiene un soporte más limitado para tipos de orden superior.

Ambos lenguajes poseen una alta **seguridad de tipos**. En cuanto a **null safety**, Scala la ofrece de forma parcial (no estricta), mientras que Rust la garantiza de forma total al no permitir valores nulos.

En cuanto al **chequeo de tipos** y la **dureza** del tipado, tanto Scala como Rust utilizan tipado estático y fuerte, además de contar con inferencia de tipos. No obstante, Scala permite más conversiones implícitas, lo que aporta mayor flexibilidad y reduce la verbosidad del código. Rust, por su parte, exige conversiones mayormente explícitas, lo que incrementa el control pero también la rigidez. En este aspecto, se suele destacar la seguridad de Rust, aunque Scala logra un buen equilibrio al favorecer la expresividad.

Respecto a los **tipos básicos**, Scala ofrece una variedad de tipos primitivos como Int, Long, Short, Byte, Float, Double, Char, String y Boolean. En Rust, estos se agrupan bajo el concepto de escalares, incluyendo enteros, números de punto flotante, booleanos y caracteres. Ambos lenguajes cubren adecuadamente las necesidades fundamentales en este nivel.

En relación con los **tipos compuestos**, Scala dispone de tuplas y una amplia variedad de colecciones como Array, List, Set y Map, generalmente inmutables por defecto, lo que favorece un estilo de programación más seguro y declarativo. Rust, en cambio, ofrece principalmente tuplas y arrays, con menor nivel de abstracción en este aspecto, lo que hace que Scala resulte más expresivo y conveniente para modelar estructuras complejas.

En los **tipos personalizados**, Scala presenta una gran riqueza mediante clases, objetos, traits y case classes, estas últimas especialmente útiles por su inmutabilidad y soporte para pattern matching. Además, permite redefinir tipos existentes mediante alias (type). Rust utiliza structs y enums, que son muy potentes, pero con un enfoque más explícito y menos orientado a la abstracción de alto nivel, donde Scala vuelve a destacarse por su flexibilidad.

En cuanto al **encapsulamiento**, Scala ofrece modificadores como private y protected, siendo público por defecto, y además permite un control más granular como private[X] o private[this]. Rust organiza el código mediante módulos y utiliza el modificador pub, siendo privado por defecto. Si bien el

enfoque de Rust es más restrictivo, Scala brinda mayor fineza en el control de visibilidad, lo que puede resultar más conveniente en diseños complejos.

En cuanto a la **abstracción**, Scala ofrece clases abstractas y traits, permitiendo definir miembros abstractos que deben ser implementados en las subclases. Además, incorpora abstracción de control mediante funciones de orden superior, lo que amplía significativamente las posibilidades expresivas del lenguaje. Por su parte, Rust utiliza traits y genéricos para definir comportamientos compartidos y favorecer la reutilización, aunque con un enfoque más explícito. En este sentido, Scala se destaca por brindar herramientas de abstracción más flexibles y de mayor nivel.

Respecto a las **reglas de conversión**, Scala permite conversiones implícitas que se aplican para adaptar tipos a los esperados, por ejemplo al convertir un `Int` a `Double` o al ajustar el receptor de una llamada. Esto aporta fluidez y reduce la necesidad de código adicional. En cambio, Rust no realiza conversiones automáticas de este tipo, requiriendo conversiones explícitas mediante mecanismos como `as`, o traits como `From` e `Into`, además de contar con coerción de desreferenciación.

Por último, en las **reglas de equivalencia**, Scala distingue entre igualdad estructural (`==`, que compara en profundidad mediante `equals`) e igualdad por referencia (`eq`). También dispone de métodos como `canEqual` y `hashCode` para un control más fino. Rust, en cambio, utiliza traits como `PartialEq` para definir igualdad en profundidad, donde cada tipo debe especificar su comportamiento.

En cuanto a **características distintivas**, Scala se destaca por contar con un sistema de tipos avanzado que incluye genéricos, traits y otros mecanismos de abstracción. En particular, los **traits** son una herramienta fundamental para la reutilización de código, ya que permiten la composición de comportamientos mediante **mixins**. Este enfoque resuelve las limitaciones de la herencia múltiple tradicional a través de un proceso de linearización, posibilitando combinar múltiples funcionalidades de manera flexible sin recurrir a jerarquías rígidas.

Por su parte, Rust presenta como rasgo diferencial su sistema de **ownership y borrowing**, el cual garantiza la seguridad de memoria sin necesidad de un recolector de basura. En este modelo, cada valor tiene un único dueño, y dicho valor se elimina automáticamente cuando su dueño sale de alcance. A su vez, el **borrowing** permite trabajar con referencias a un valor sin transferir su propiedad, utilizando referencias inmutables (`&`) o mutables (`&mut`). Este sistema está regido por reglas estrictas: en un momento dado, solo puede existir una referencia mutable o múltiples referencias inmutables, pero no ambas simultáneamente, y además todas las referencias deben ser siempre válidas. Este enfoque asegura un control preciso de los recursos y previene errores comunes en tiempo de ejecución.

Por el contrario, Scala logra un equilibrio entre seguridad y productividad, apoyándose en un sistema de tipos avanzado, mecanismos de abstracción de alto nivel y una sintaxis más concisa. Esto permite modelar soluciones de manera más declarativa y reducir la cantidad de código necesario, favoreciendo, como fue mencionado anteriormente, la legibilidad y la mantenibilidad a largo plazo.

En este sentido, aunque Rust resulta una opción más adecuada en contextos donde el rendimiento y el control fino de la memoria son críticos, Scala se posiciona como una alternativa más conveniente para el desarrollo general de aplicaciones, donde la rapidez de desarrollo, la expresividad y la capacidad de abstracción juegan un rol central.

Control de Flujo

Scala evalúa sus expresiones de izquierda a derecha. Además las asignaciones son expresiones que retornan "()", mencionados como tipo "Unit" que se puede comparar como similar a "void" en lenguajes como Java.

Dado que Scala soporta tanto el paradigma OO como el funcional se hace una distinción en cómo se tratan los efectos colaterales:

- En el caso OO los efectos colaterales ocurren
- En el caso funcional se busca fuertemente la **transparencia referencial**

También admite también los cuatro tipos de evaluaciones:

- *Estricta*: por default suele ser la forma estándar de evaluar.
- *No-estricta*: Scala permite esta forma de evaluación con los llamados parámetros por nombre (by-name parameters) permitiendo que éstos se evalúen cuando son referenciados.
- *Perezosa*: Scala proporciona el modificador "lazy" permitiendo que las variables declaradas de esta forma se inicialicen únicamente cuando son referenciadas y no se vuelve a actualizar su valor.
- *Corto-circuito*: Soporta el uso de los operandos "&&" y "||" en expresiones.

Rust evalúa sus expresiones de izquierda a derecha y al igual que Scala sus asignaciones son expresiones que devuelven "()". No obstante, Rust no soporta los cuatro tipos de evaluación de expresiones. Siendo únicamente:

- *Estricta*: al igual que Scala, es el comportamiento por default
- *Perezosa*: únicamente en iterators
- *Cortocircuito*: lo admite, al igual que Scala, con operandos booleanos (mediante uso de "&&" y "||")

Estructuras de control

Rust

- If-else: Es una expresión, retorna siempre como valor la última línea de los bloques que posea
- Loop: repetición incondicional, pudiendo salir de la misma a través de break (puede retornar al salir de este)
- for: no es una expresión, sirve para iterar sobre cualquier cosa que implemente "iterator"
- while: repetición condicional, no se considera una expresión
- match: es una expresión

Scala

if, while, for(sin-yield), for-(con-yield), try, match (similar a switch), llamadas a funciones

ejemplo:

```
val filename = if (!args.isEmpty) args(0) else "default.txt"
```

if como expresión en Scala y retorno de cada bloque

Algo **muy positivo de Scala** es que permite escribir estructuras de control nuevas gracias a las funciones de primera-clase. Como éstas últimas son esencialmente valores como cualquier otro, admiten ser usadas, entre otras formas, como parámetros. Se puede entonces abstraer comportamiento como el siguiente:

```
def twice(op: Double => Double, x: Double) =  
    op(op(x))
```

frente a un uso de la función como:

```
twice(_ + 1, 5)
```

Viendo el cuerpo de twice “op(op(x))” de **adentro hacia afuera** resulta en `_ + 1`, donde “_” se instancia con “5” dado que es el valor de x en la llamada de twice (resulta 6) `op(op(x))`: 6 + 1 donde op(x) retornó “6” y da como resultado 7

Esta simple función nuclea un comportamiento en la ejecución que es realizar una operación dos veces, pudiendo instanciarse de distintas formas (resta, división, etc)

En líneas generales ambos lenguajes presentan similitudes en las estructuras de control que proporcionan. Además algunas de ellas son consideradas expresiones por lo que admiten ser utilizadas del lado derecho de una asignación. Una distinción importante en ambos casos es que las asignaciones si bien son expresiones (retornan “()”) no tienen ninguna utilidad real en tanto su “retorno”. Esto último genera que “facilidades” de escritura como en **C** del tipo: `while((c = getchar()) != '$')` no sean realmente útiles ya que comparar el tipo “()” resulta en un comportamiento que nada tiene que ver con la semántica del ejemplo de C. Esto conlleva a separar las asignaciones de la lógica misma en el uso de las estructuras de control tanto en Rust como en Scala.

Elección del Lenguaje

Por lo desarrollado anteriormente observamos que Rust brilla en características de seguridad, manejo de memoria, eficiencia pero a costa de ello pierde flexibilidad y abstracción. Por el contrario, Scala se centra en escalabilidad, modularidad y abstracción presentando elementos para facilitar estas características y logrando también mayor flexibilidad. Por todas las ventajas que mencionamos en todas las secciones anteriores decidimos elegir a Scala como lenguaje para el desarrollo de este proyecto.

Anexo

Prioridad	Aspectos importantes para este proyecto
● Críticos	Librerías / ecosistema, Procesamiento y transformación de strings y datos, Modularidad, Manejo de errores, Velocidad de desarrollo
● Muy importantes	Legibilidad, Nivel de abstracción
● Importantes	Tipado, Testing, Deployment / CLI, Concurrencia
● Poco importantes	Performance extrema, Uso de memoria, Seguridad de bajo nivel

Uso de IA

En el desarrollo de este trabajo se utilizaron modelos de lenguaje (LLMs) como herramientas de asistencia. Fueron empleados como apoyo en la búsqueda y síntesis de información dentro de la documentación oficial de Rust y Scala, dado el volumen y la densidad técnica de dichos manuales.

En ambos casos, el uso de estas herramientas fue complementario al criterio y al trabajo del equipo, siendo todo el contenido revisado, ajustado y validado por los integrantes del grupo.

NotebookLM:

I'd like to see where in both texts there's solid information about control flow in both Scala and Rust. Particularly:

- . How are expressions evaluated (right to left, or left to right)
- . Can it be collateral effects
- . The expressions in each language use: strict evaluation, non-strict evaluation, lazy evaluation, short-circuit evaluation?

Please provide the pages of the sources you find in each book along with your answer

Tell me exactly in which section of the Rust programming language manual gives a clear definition of the language's paradigm

In both the Scala and Rust assignments are expressions. They both return "()" from what I've seen, find if there's a key language's design decision regarding this and if so return the page and text where this is highlighted

ChatGPT:

"Scala has only a handful of built-in control structures. The only control structures are if, while, for, try, match, and function calls. The reason Scala has so few is that it has included function literals since its inception. Instead of accumulating one higher-level control structure after another in the base syntax, Scala accumulates them in libraries" what does this actually mean?
